Chapter 4
Enhanced Entity-Relationship

PROBLEM with ER notation

The entity relationship model in its original form did not support the **specialization/ generalization** abstractions

Extended Entity-Relationship (EER) Model

- Incorporates **Set-subset relationships**
- Incorporates **Specialization/Generalization** Hierarchies

This chapter illustrates how the ER model can be extended with

- Set-subset relationships and Specialization/Generalization Hierarchies and how to display them in EER diagrams

Enhanced-ER (EER) Model Concepts

- Includes all modeling concepts of basic ER
- **Additional concepts**: subclasses/superclasses, specialization/generalization, categories, attribute inheritance
- The resulting model is called the Enhanced-ER or Extended ER (E2R or EER) model
- It is used to model applications more **completely** and **accurately** if needed
- It includes some **object-oriented** concepts, such as **inheritance**

Subclasses and Superclasses (1)

- An entity type may have <u>additional meaningful sub groupings</u> of its entities
- Example: EMPLOYEE may be further grouped into SECRETARY, ENGINEER, MANAGER, TECHNICIAN, SALARIED_EMPLOYEE, HOURLY_EMPLOYEE,…
  - Each of these groupings is a subset of EMPLOYEE entities
  - Every entity that is a member of one of this sub grouping is also an EMPLOYEE
  - Each is called a subclass of EMPLOYEE
  - EMPLOYEE is the superclass for each of these subclasses
- These are called **superclass/subclass** relationships.
- Example: EMPLOYEE/SECRETARY, EMPLOYEE/TECHNICIAN

Subclasses and Superclasses (2)

- These are also called IS-A relationships (SECRETARY IS-A EMPLOYEE, TECHNICIAN IS-A EMPLOYEE, …).
- Note: An entity that is member of a subclass represents the **same real-world entity** as some member of the superclass
  - The Subclass member is the same entity in a distinct specific role
  - An entity cannot exist in the database merely by being a member of a subclass; it must also be a member of the superclass
  - A member of the superclass can be optionally included as a member of any number of its subclasses
- Example: A salaried employee who is also an engineer belongs to the two subclasses ENGINEER and SALARIED_EMPLOYEE
  - It is not necessary that every entity in a superclass be a member of some subclass

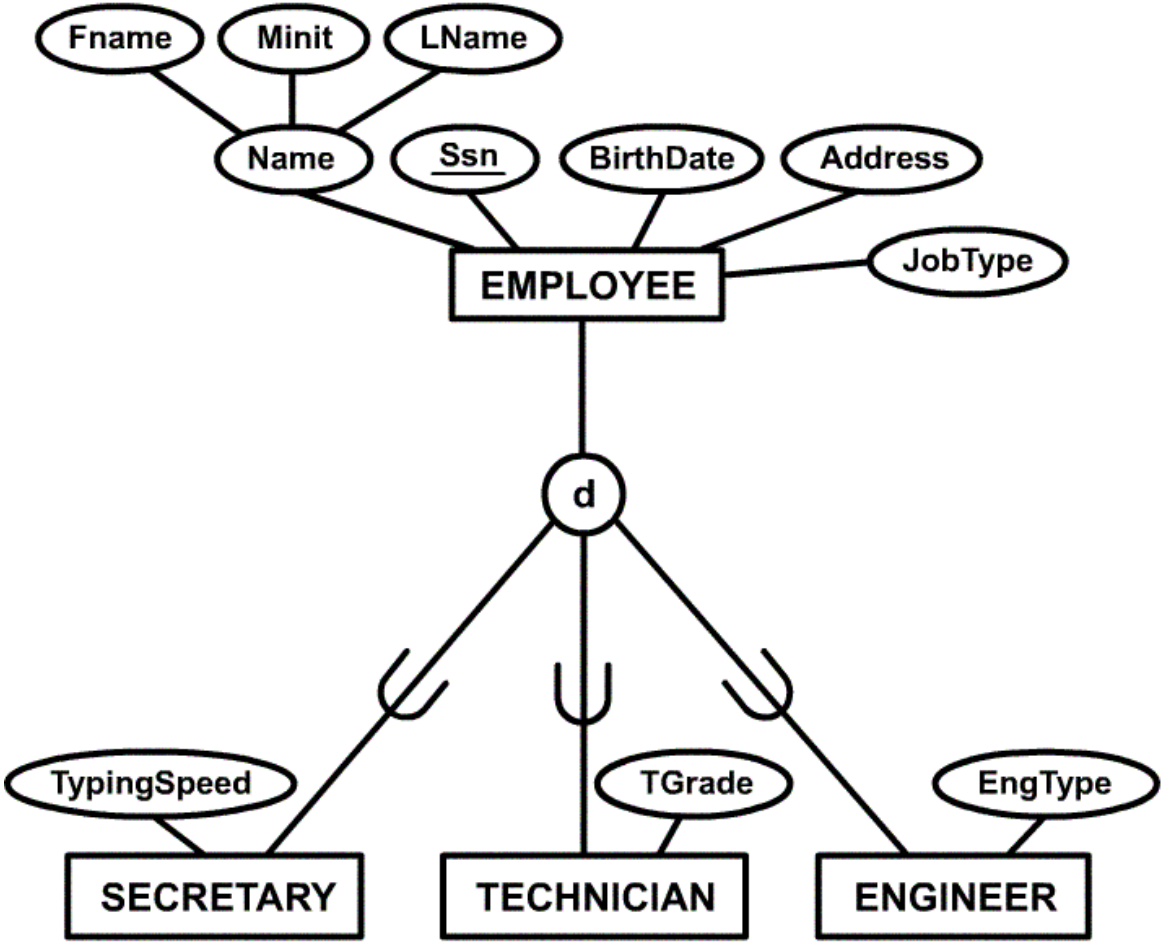
Attribute Inheritance in Superclass / Subclass Relationships

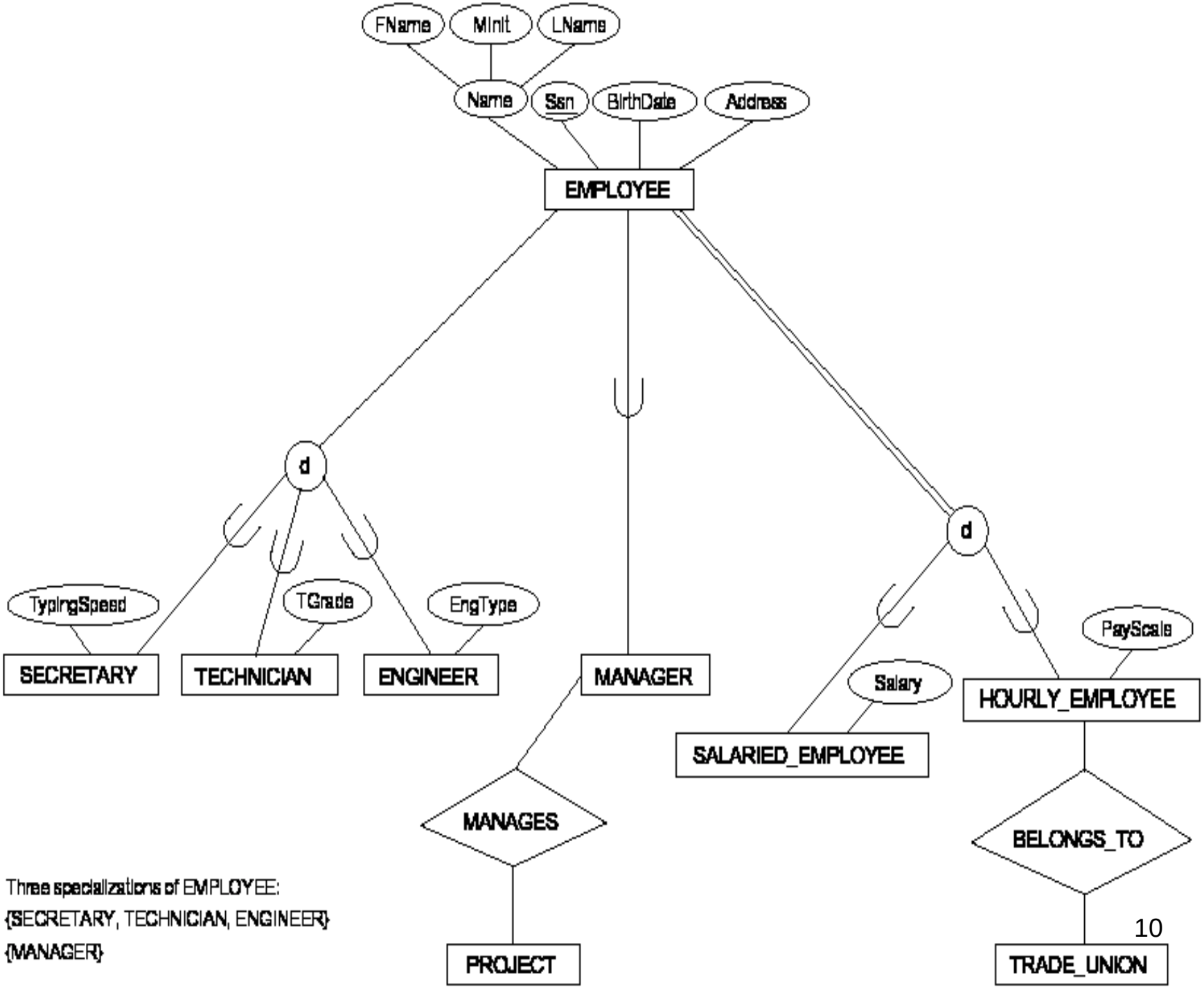
- An entity that is member of a subclass *inherits* all attributes of the entity as a member of the superclass
- It also inherits all relationships

Specialization

- Is the process of **defining a set of subclasses** of a superclass
- The set of subclasses is based upon some **distinguishing characteristics** of the entities in the superclass
- Example: {SECRETARY, ENGINEER, TECHNICIAN} is a specialization of EMPLOYEE based upon ***job type***.
  - May have several specializations of the same superclass
- Example: Another specialization of EMPLOYEE based in ***method of pay*** is {SALARIED_EMPLOYEE, HOURLY_EMPLOYEE}.
  - Superclass/subclass relationships and specialization can be diagrammatically represented in EER diagrams
  - Attributes of a subclass are called **specific attributes**. For example, TypingSpeed of SECRETARY
  - The subclass can participate in **specific relationship types**. For example, BELONGS_TO of HOURLY_EMPLOYEE

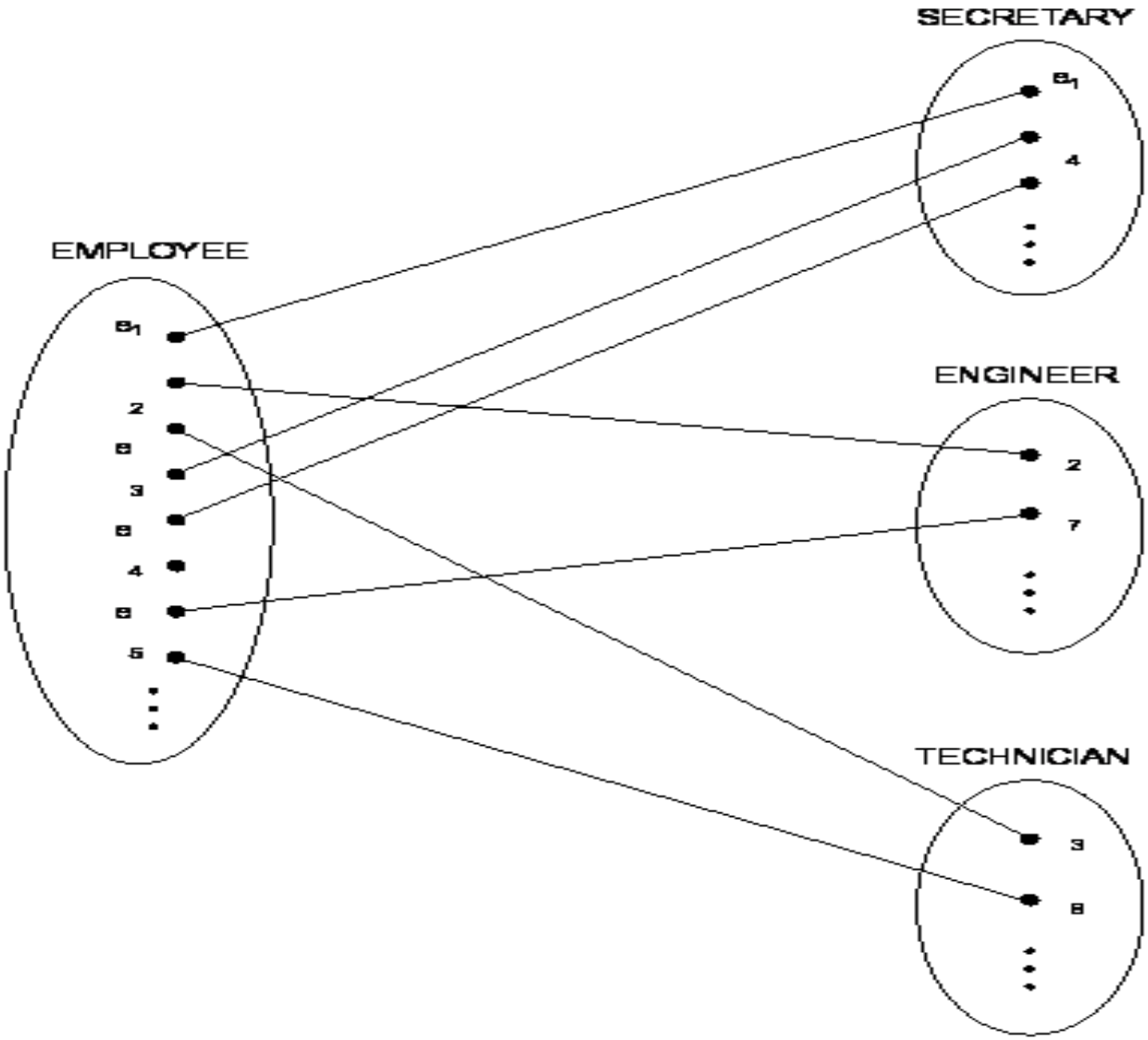
Example of a Specialization





Three specializations of EMPLOYEE:
{SECRETARY, TECHNICIAN, ENGINEER}
{MANAGER}

Instances of the Specialization



- Two **reasons** for including class/subclass relationships and specializations in a data model
  - Certain attributes may apply to some but not all entities of the superclass.
    - A subclass is defined in order to group the entities **to which these attributes apply**.
    - The members of the subclass may still share the **majority of their attributes** with the other members of the superclass.

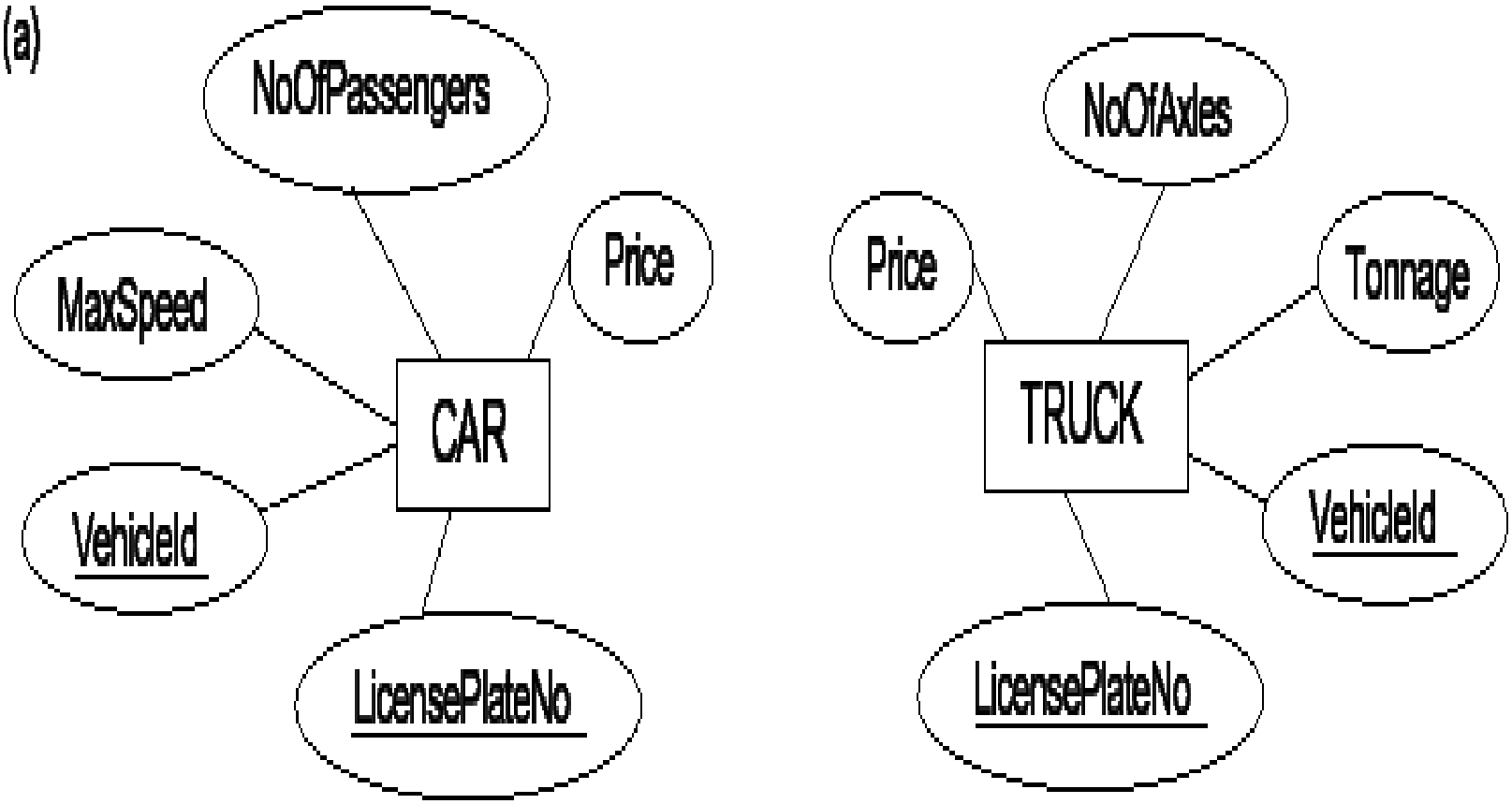
- Some relationship types may be participated in only entities that are members of the subclass.

- The specialization process allows us to
  - Define a **set of subclasses** of an entity type
  - Establish additional **specific attributes** with each subclass
  - Establish additional **specific relationship** types between each subclass and other entity types or other sub classes.

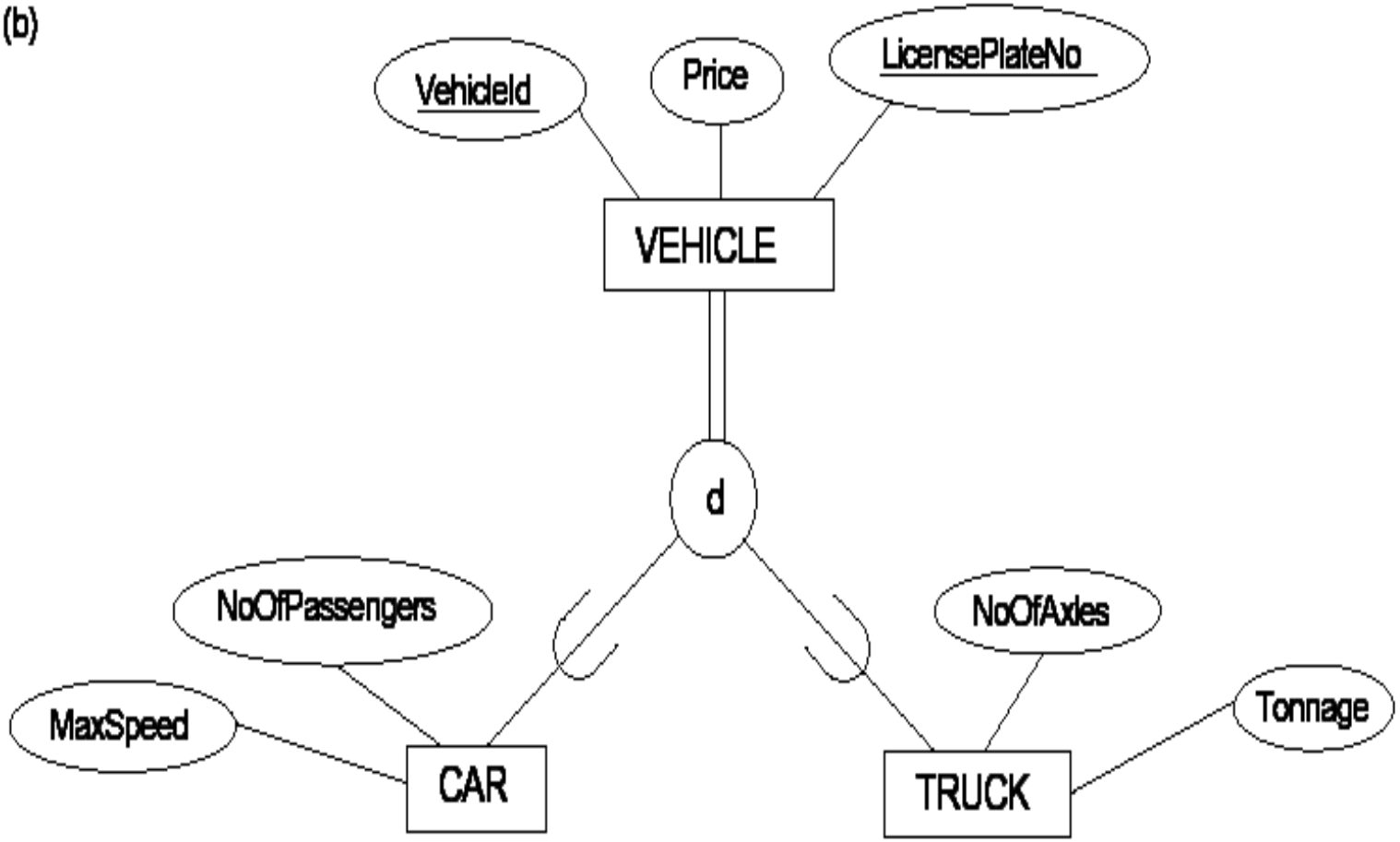
Generalization

- The reverse of the specialization process
- Several classes with common features are generalized into a superclass; original classes become its subclasses
- Example: CAR, TRUCK generalized into VEHICLE; both CAR, TRUCK become subclasses of the superclass VEHICLE.
  - We can view {CAR, TRUCK} as a specialization of VEHICLE
  - Alternatively, we can view VEHICLE as a generalization of CAR and TRUCK

(a)


NoOfPassengers
MaxSpeed
Price
CAR
VehicleId
LicensePlateNo
NoOfAxles
Price
Tonnage
TRUCK
VehicleId
LicensePlateNo

(b)

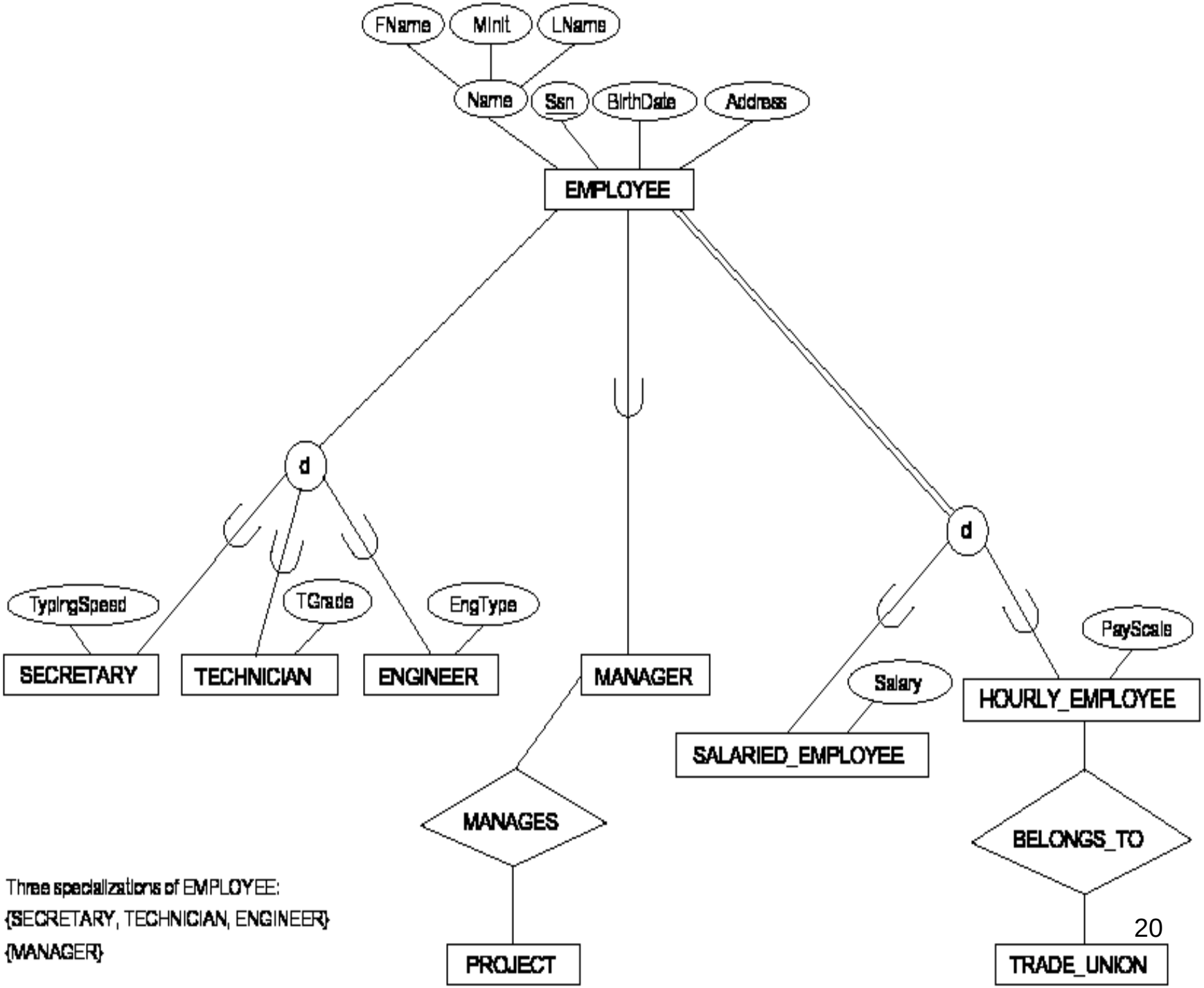

VehicleId
Price
LicensePlateNo
VEHICLE
d
NoOfPassengers
MaxSpeed
CAR
NoOfAxles
Tonnage
TRUCK

Generalization and Specialization

- Data Modeling with Specialization and Generalization
  - A superclass or subclass represents a **set of entities**
  - Shown in rectangles in EER diagrams (as are entity types)
  - Sometimes, all entity sets are simply called **classes**, whether they are entity types, superclasses, or subclasses

Constraints on Specialization and Generalization

- In general we may have **several specializations** defined on the same entity type.
- Entities may belong to subclasses in each of the specializations
- A specialization **may** also consists of a single class only

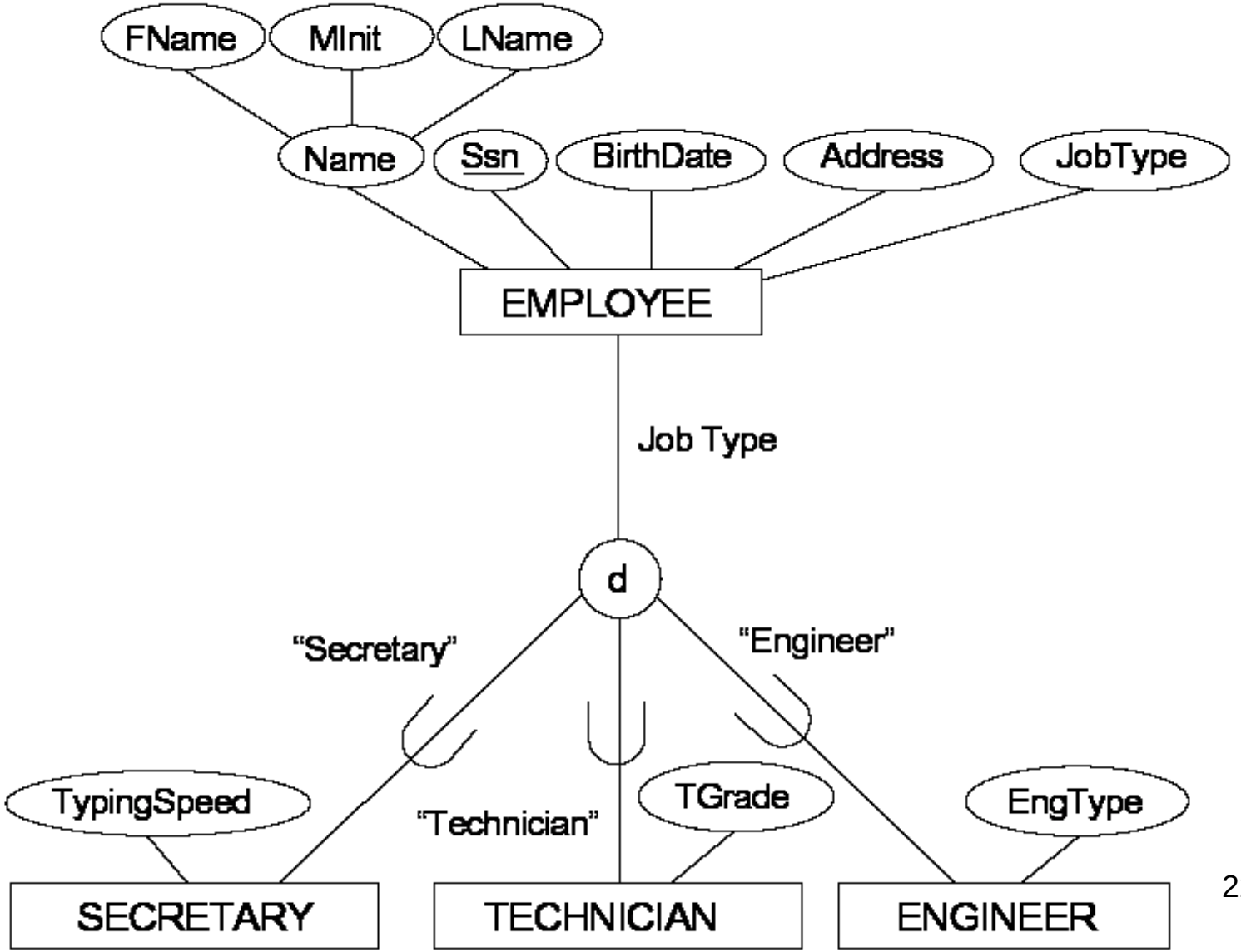


Three specializations of EMPLOYEE:
{SECRETARY, TECHNICIAN, ENGINEER}
{MANAGER}

Constraints on Specialization and Generalization (1)

- If we can **determine exactly those entities** that will become members of each subclass by a condition, the subclasses are called ***predicate-defined*** (or condition-defined) subclasses
  - Condition is a constraint that determines subclass members
  - Display a predicate-defined subclass by writing the **predicate condition next to the line attaching the subclass to its superclass**
- If all subclasses in a specialization have membership condition on same attribute of the superclass, specialization is called an ***attribute defined*-specialization**
  - Attribute is called the defining attribute of the specialization
  - Example: JobType is the defining attribute of the specialization {SECRETARY, TECHNICIAN, ENGINEER} of EMPLOYEE
- If no condition determines membership, the subclass is called ***user-defined***
  - Membership in a subclass is determined by the database users by applying an operation to add an entity to the subclass
  - Membership in the subclass is specified individually for each entity in the superclass by the user

An attribute-defined specialization on the JobType attribute of EMPLOYEE.



Constraints on Specialization and Generalization (2)

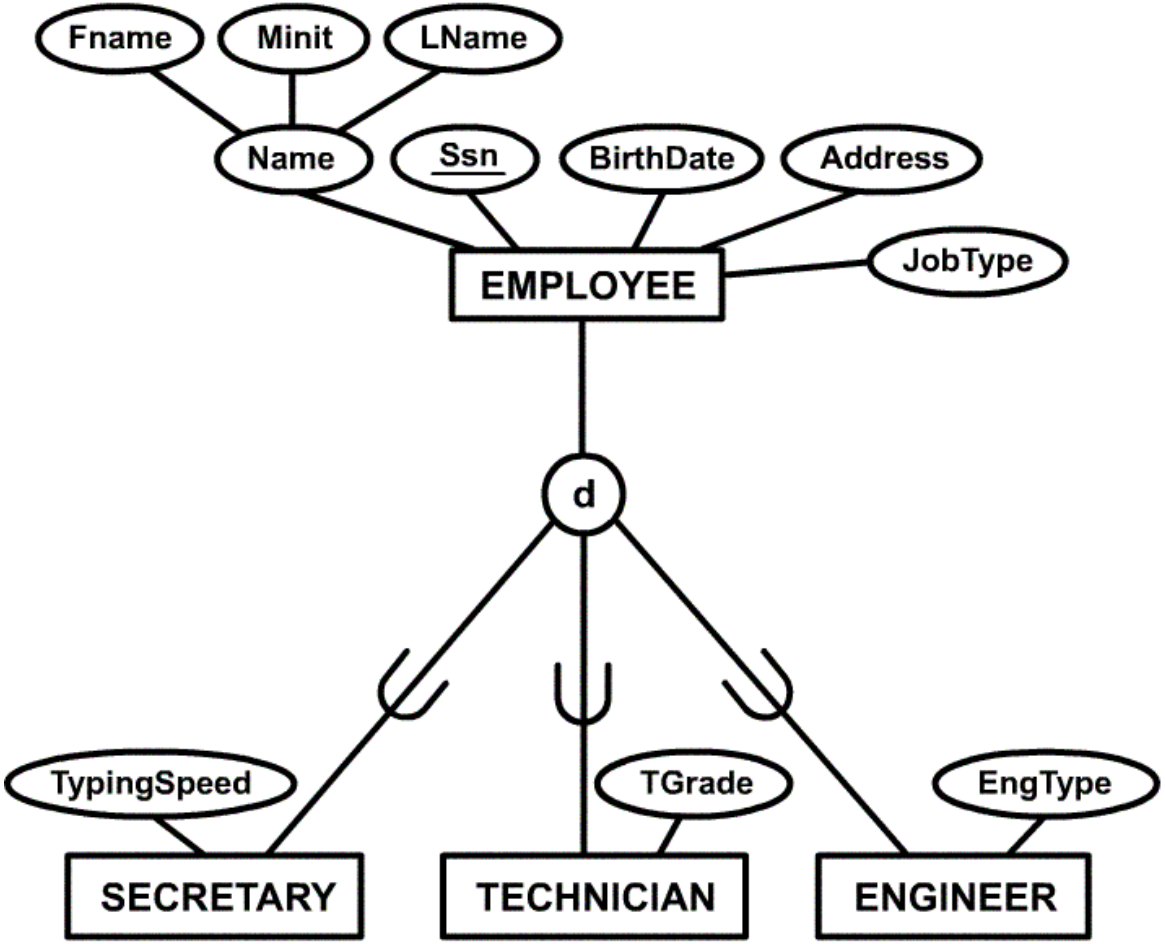
- Two other conditions apply to a specialization/generalization:
- **<u>Disjointness Constraint</u>**:
  - Specifies that the subclasses of the specialization must be **disjointed** (an entity can be a member of at most one of the subclasses of the specialization)
  - Specified by **d** in EER diagram
  - If not disjointed, **overlap**; that is the same entity may be a member of more than one subclass of the specialization
  - Specified by **o** in EER diagram
- **<u>Completeness Constraint</u>**:
  - **Total** specifies that **every entity** in the superclass must be a member of some subclass in the specialization/ generalization
  - Shown in EER diagrams by a double line
  - **Partial** allows **an entity not to belong** to any of the subclasses
  - Shown in EER diagrams by a single line

Constraints on Specialization and Generalization (3)

- Hence, we have four types of specialization/generalization:
  - Disjoint, total
  - Disjoint, partial
  - Overlapping, total
  - Overlapping, partial

- Note: **<u>Generalization usually is total</u>** because the superclass is derived from the subclasses.

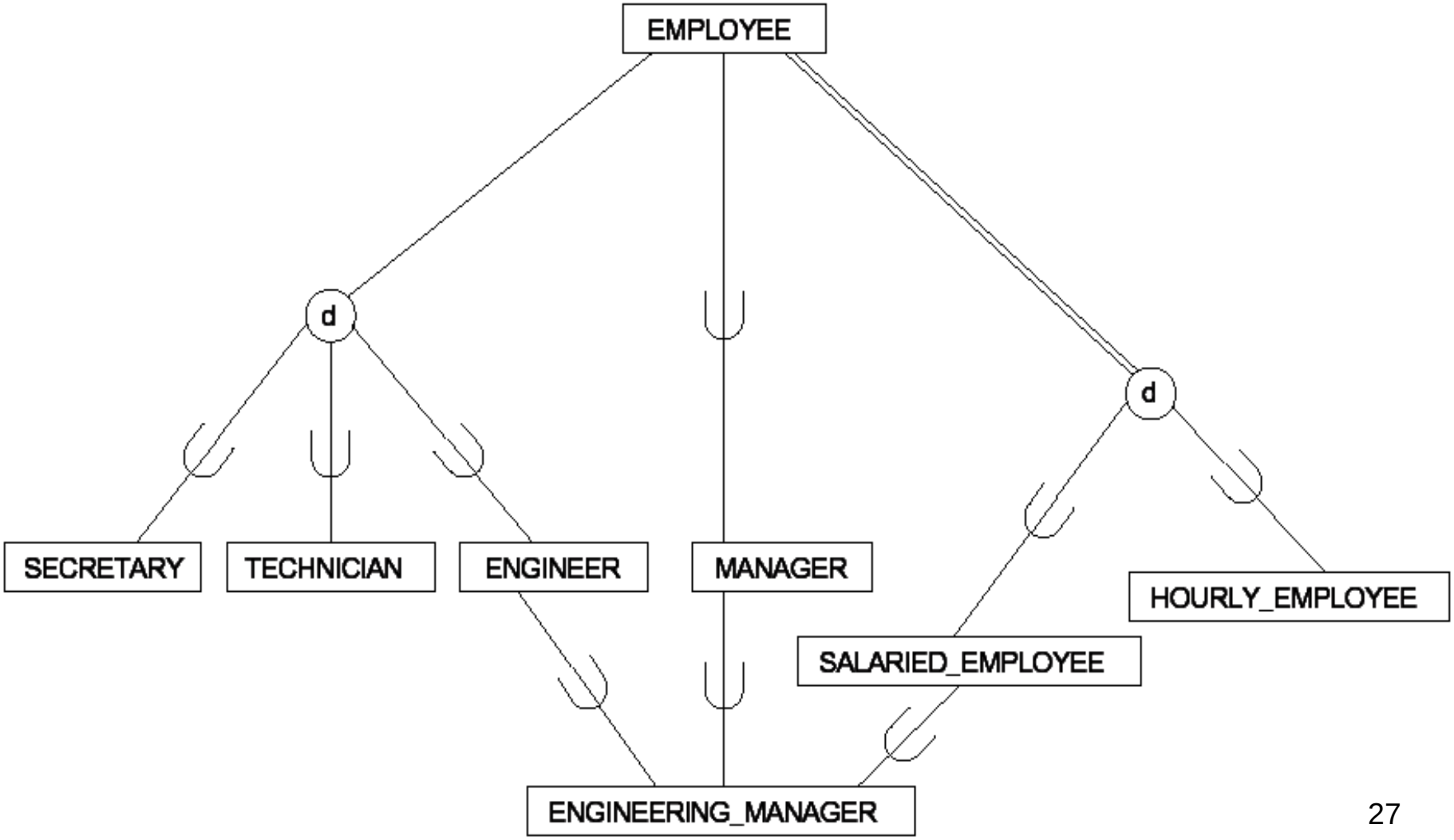
Example of disjoint partial Specialization



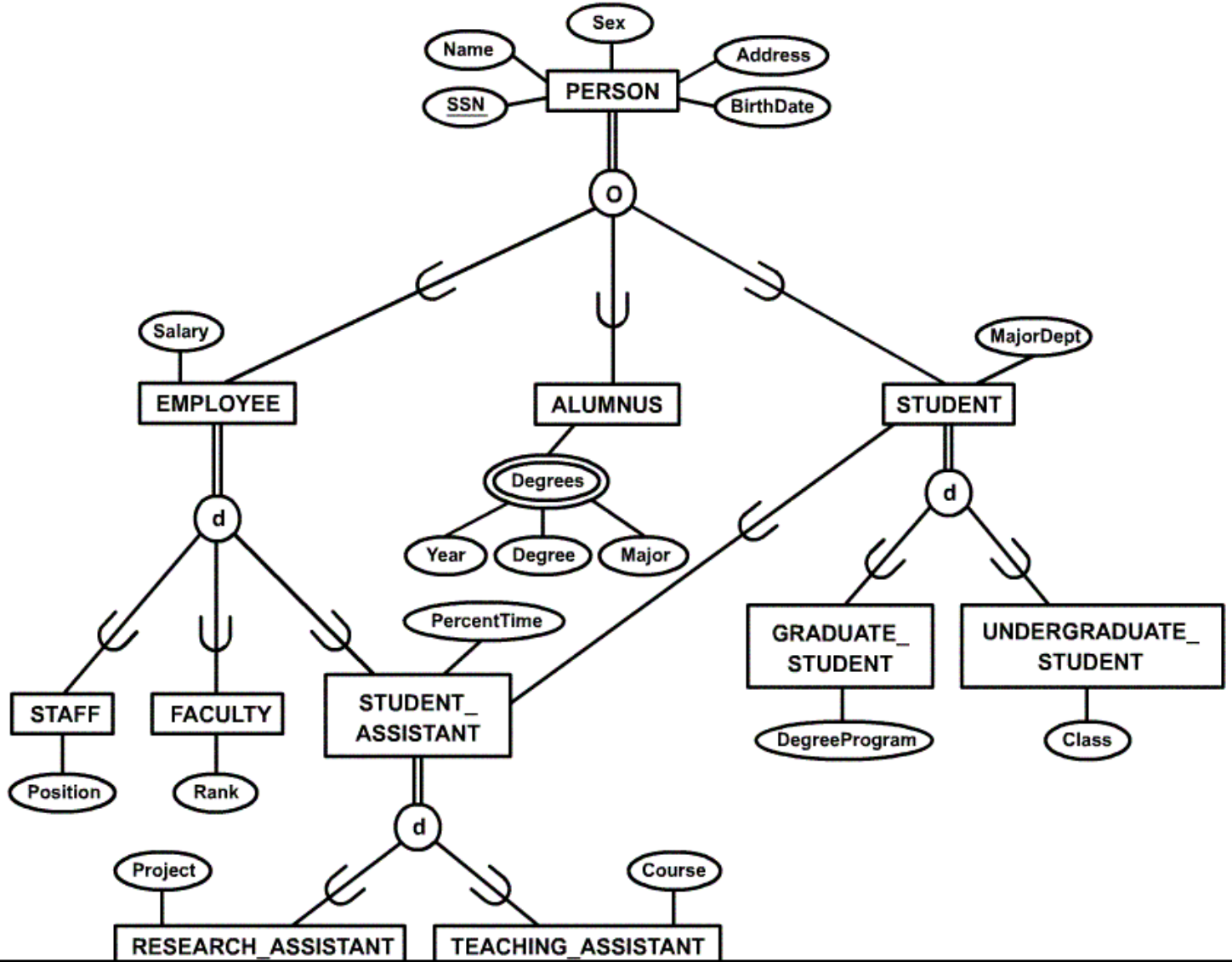
Specialization / Generalization Hierarchies, Lattices and Shared Subclasses

- A subclass may itself have **further subclasses** specified on it
- Forms a **hierarchy or a lattice**
- **Hierarchy** has a constraint that every subclass has **only one superclass** (called *single inheritance*)
- In a **lattice**, a subclass can be subclass of **more than one superclass** (called *multiple inheritance*)
- In a lattice or hierarchy, a subclass inherits attributes not only of its direct superclass, but also of all its **predecessor** superclasses
- A subclass with **more than one superclass** is called a **shared subclass**
- Can have specialization hierarchies or lattices, or generalization hierarchies or lattices
- In **specialization**, start with an entity type and then define subclasses of the entity type by successive specialization (**top down** conceptual refinement process)
- In **generalization**, start with many entity types and generalize those that have common properties (**bottom up** conceptual synthesis process)
- In practice, the combination of two processes is employed

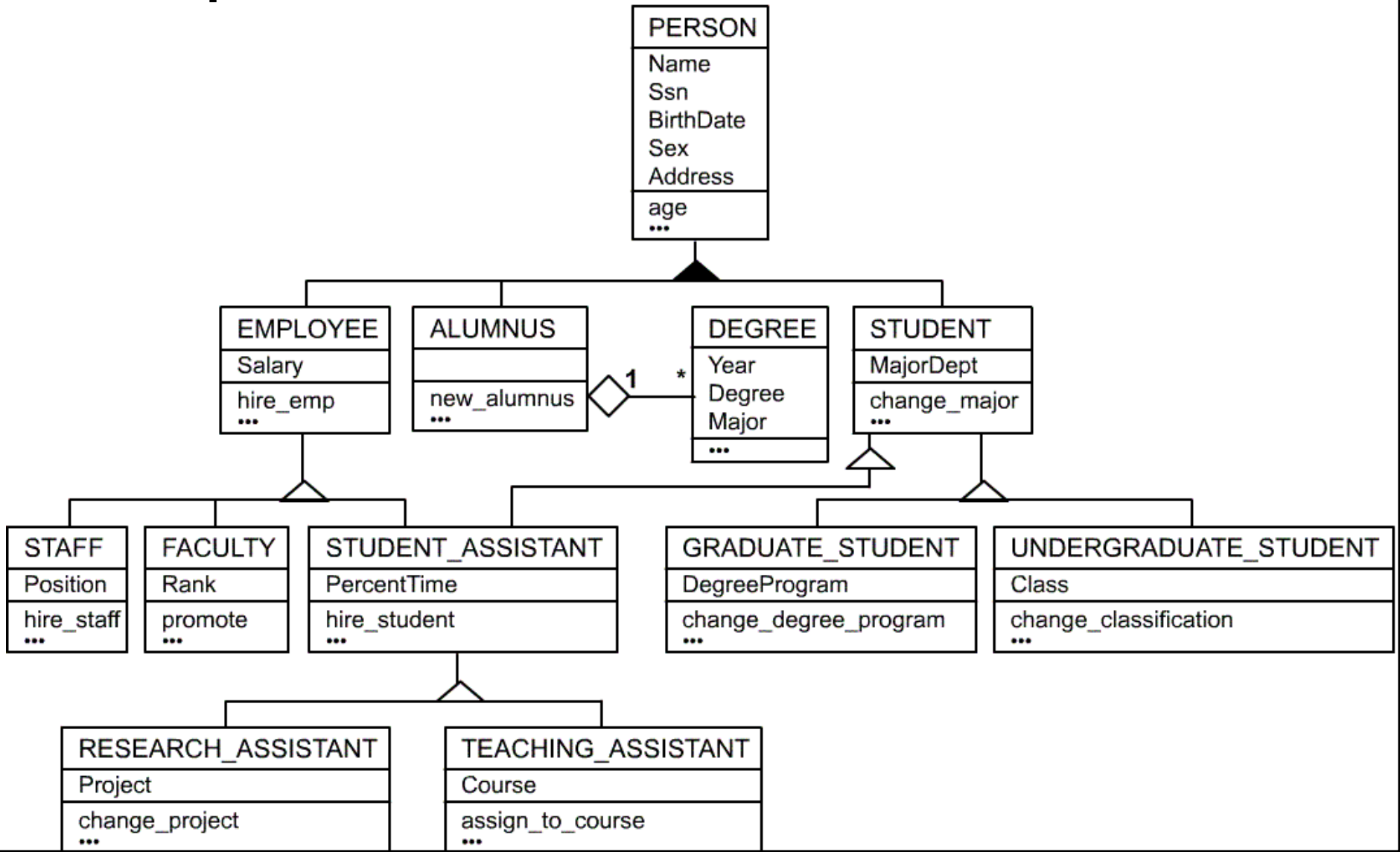
A specialization lattice with the shared subclass ENGINEERING_MANAGER.



Specialization / Generalization Lattice Example (UNIVERSITY)

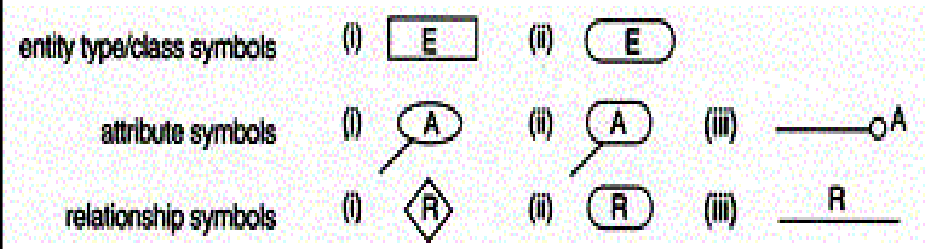


UML Example for Displaying Specialization / Generalization

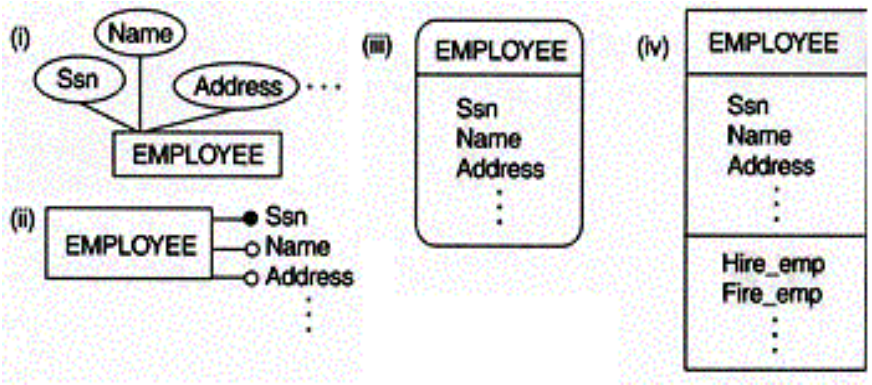


Alternative Diagrammatic Notations

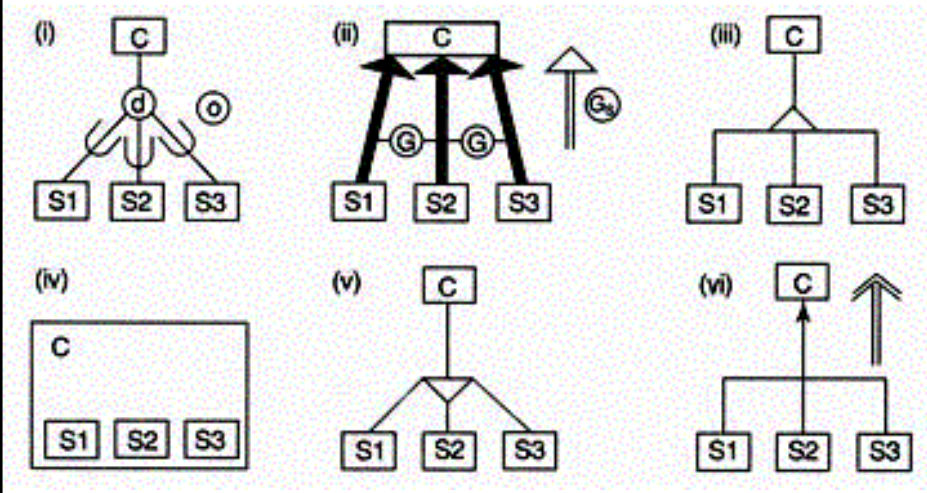
Symbols for entity type / class, attribute and relationship



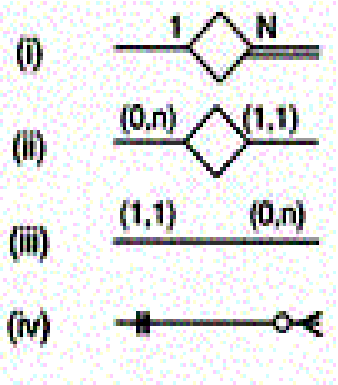
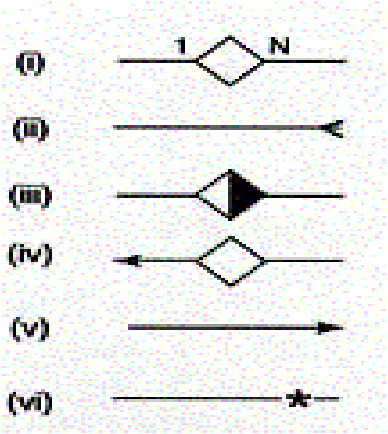
Displaying attributes



Notations for displaying specialization / generalization



Various (min, max) notations



Displaying cardinality ratios